

CO2 ASSESSMENT TOOL 3

CSA1305-TOC

V.Navyasree(192373050)

Question 1

A company is developing a smart security system...

Introduction

Modern organizations use smart security systems to control access to restricted areas such as laboratories, banks, airports, and research centers. Every employee entering the laboratory uses a digital access card. The system validates the access sequence before allowing entry.

The alphabet is

where

- $a \rightarrow$ Valid ID Scan
- $b \rightarrow$ Biometric Verification
- $c \rightarrow$ Door Access Confirmation

The security rules are:

- Every sequence must start with a
- It may contain zero or more b
- It must end with c

Examples:

Accepted

ac

abc

abbc

abbbc

Rejected

bc

aac

cab

ab

Step 1: Regular Expression

Since

- first symbol must be a
- followed by zero or more b
- ending with c

the regular expression becomes

Explanation

- $a \rightarrow$ mandatory
- $b^* \rightarrow$ any number of biometric checks
- $c \rightarrow$ mandatory ending

Step 2: DFA Design

States

q0

Start state

Waiting for ID card.

q1

ID successfully scanned.

q2

Biometric verification.

The employee may verify fingerprints multiple times.

q3

Door opens.

Accept state.

qd

Dead state.

Invalid sequence.

Transition Table

Current State	Input a	Input b	Input c
→ q0	q1	qd	qd
q1	qd	q2	q3
q2	qd	q2	q3
★ q3	qd	qd	qd
qd	qd	qd	qd

DFA Explanation

Example

Input

abbbc

Processing

q0 --a--> q1

q1 --b--> q2

q2 --b--> q2

q2 --b--> q2

q2 --c--> q3

Accepted

Time Complexity

The DFA scans each symbol only once.

Therefore

where n is the input length.

Advantages

- Fast authentication
- Constant memory

- Low hardware cost
- Suitable for embedded systems
- Secure access validation

Real-life Applications

- Airport entry
- Smart locks
- RFID systems
- ATM authentication
- Employee attendance systems

Conclusion

The DFA recognizes only valid access sequences and rejects invalid ones efficiently.

Question 2

Minimize the DFA

Introduction

A DFA may contain unnecessary states.

These unnecessary states increase

- memory
- processing time
- hardware complexity

DFA minimization removes equivalent states.

Steps

Step 1

Remove unreachable states.

Example

q5

cannot be reached.

Remove it.

Step 2

Separate

Final states

and

Non-final states.

Step 3

Compare transitions.

Equivalent states are merged.

Step 4

Draw minimized DFA.

Why Minimize?

Suppose

Original DFA

10 states

After minimization

6 states

Benefits

- 40% less memory
- faster execution
- fewer flip-flops
- lower power

Embedded Systems

Examples

- Smart watches
- ATM
- IoT devices
- Medical devices

All require minimum hardware.

Advantages

- ✓ Lower memory
- ✓ Less hardware
- ✓ Faster execution
- ✓ Less power consumption
- ✓ Easier implementation

Conclusion

Minimized DFA performs exactly the same task with fewer states.

Question 3

Convert Finite Automaton into Regular Expression

Introduction

Regular Expressions and Finite Automata are equivalent computational models.

Every DFA/NFA has an equivalent Regular Expression.

State Elimination Method

Step 1

Identify

Start state

Accept state

Step 2

Remove intermediate states one by one.

Step 3

Update transitions.

Step 4

Continue until only

Start

Accept

remain.

Step 5

Remaining edge label

=

Regular Expression

Importance

Regular expressions

- compress DFA
- easier to understand
- easier to implement

Applications

- Compiler lexical analyzer
- Pattern matching
- Search engines
- Virus scanning
- Spam filtering

Advantages

Compact representation

Easy implementation

Efficient matching

Conclusion

Finite Automata and Regular Expressions recognize exactly the same Regular Languages.

Question 4

Grammar

$S \rightarrow aSc | T | U | \epsilon$

$T \rightarrow bTc | \epsilon$

$U \rightarrow aUb | \epsilon$

(A) Language

Grammar produces

$a^n c^n$

$b^n c^n$

$a^n b^n$

including

ϵ

Therefore

(B)

Word

aaabbc

Cannot be generated.

Reason

Grammar never produces

aaa

bb

c

in this order.

Hence

No derivation tree.

(C)

Ambiguous Grammar

A grammar is ambiguous if

one string has

two different parse trees.

Here

each production generates unique strings.

Therefore

Grammar is

Not Ambiguous.

Applications

- Programming language parsing
- XML
- SQL
- JSON
- Compiler syntax analysis

Conclusion

The grammar generates three independent language families.

Question 5(i)

Closure Property

Statement

If

L_1

and

L_2

are regular

then

is also regular.

Proof

Suppose

M_1 recognizes L_1

M_2 recognizes L_2

Construct Product DFA

States

$Q_1 \times Q_2$

Accept if

either DFA accepts.

Hence

Union is Regular.

Other Closure Operations

Regular Languages are closed under

- Union
- Intersection
- Complement
- Difference
- Concatenation
- Kleene Star
- Reversal

Applications

Firewall

Spam detection

Intrusion detection

Pattern matching

Compiler scanners

Conclusion

Closure properties allow combining multiple rules without losing regularity.

Question 5(ii)

Is

Regular?

Answer

No.

Pumping Lemma

Assume

Language is Regular.

Take

a^q

where

q

is prime.

Split

xyz

Pump

xy^2z

Length becomes

$q+k$

or for a suitable pumping choice, a composite length is obtained.

The resulting string no longer has a prime number of a 's.

Contradiction.

Therefore

Language is

Not Regular.

Why?

Finite Automata

cannot count
prime numbers.
They have
finite memory.

Applications

Requires

- Pushdown Automata
- Turing Machines
- Context-sensitive computation

Conclusion

The language of prime-length strings cannot be recognized by finite automata because it requires unbounded counting beyond the capability of regular languages.